UNIVERSIDAD SURCOLOMBIANA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SOFTWARE

E03 - Diseño del Producto y de la Solución

Reto de Agroindustria Cacaotera

Integrantes del Equipo:

Brayan Toro Bustos
David Ricardo Graffe
Alejandro Puentes Salazar
Emanuel González Hio

Fecha: 16 / 10 / 2026

Versión 1.0

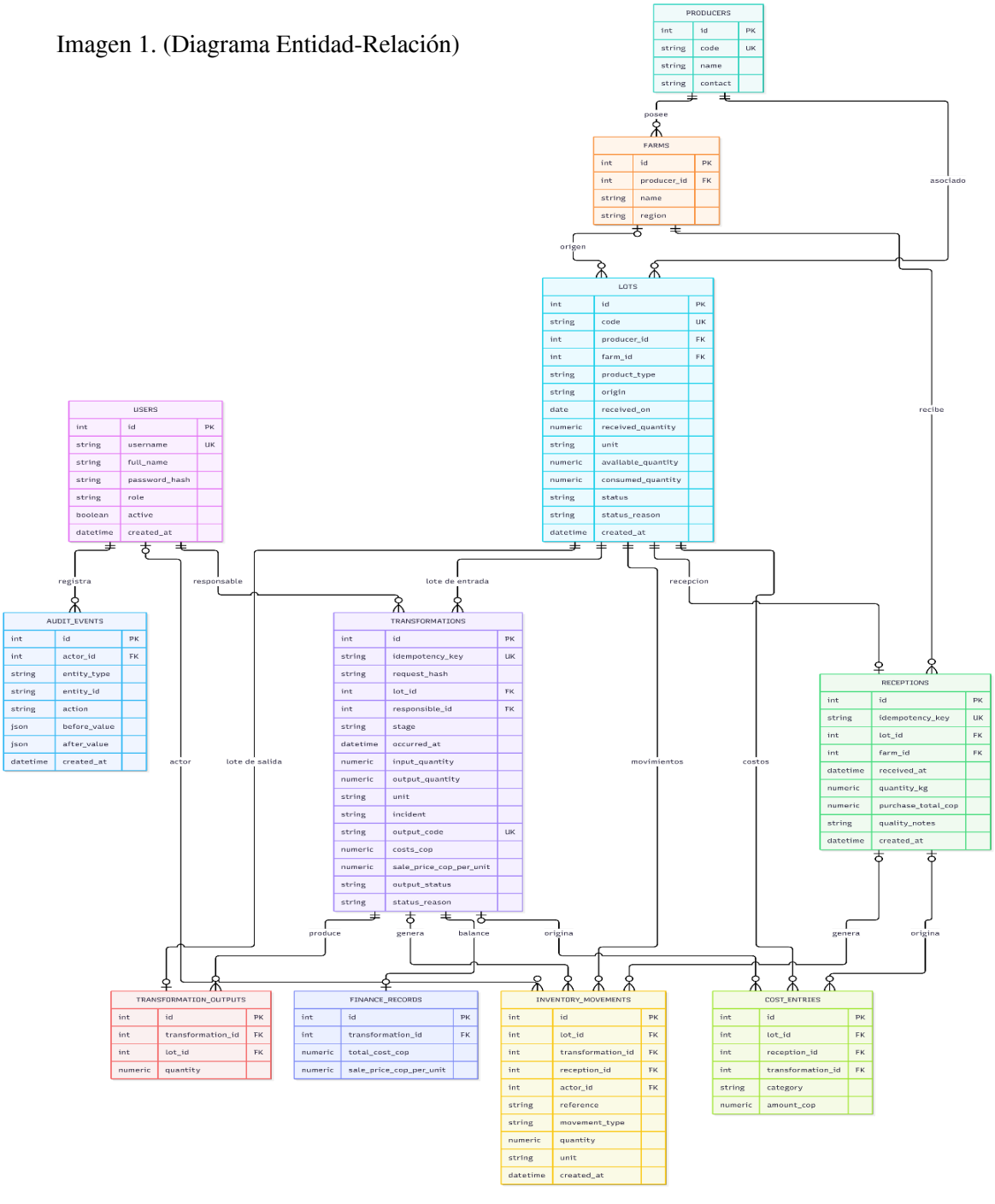
1. Diagrama de Arquitectura y Componentes

El sistema se estructura en una arquitectura cliente-servidor de tres capas, orquestada mediante contenedores Docker para garantizar un despliegue reproducible.

- **Frontend (Cliente):** Desarrollado en React con TypeScript. Se encarga de la interfaz de usuario (Operador, Administrador, Comprador) y de la captura de datos. Implementa almacenamiento local (IndexedDB) para soportar conectividad limitada, permitiendo encolar operaciones de recepción temporalmente.
- **Backend (Servidor):** API RESTful desarrollada en FastAPI (Python). Centraliza y aplica estrictamente todas las reglas de negocio, cálculos analíticos (mermas, costos) y el control de acceso.
- **Capa de Datos:** Motor de base de datos relacional PostgreSQL, gestionado a través del ORM SQLAlchemy con control de migraciones mediante Alembic.

2. Modelo de Datos (Entidades, Claves y Relaciones)

Imagen 1. (Diagrama Entidad-Relación)



3. Reglas de Negocio, Estados y Control de Acceso

Para garantizar la fiabilidad del sistema y cumplir con las restricciones del reto, las validaciones de seguridad e inventario no dependen exclusivamente de la interfaz visual (React), sino que se aplican de manera estricta en la capa del servidor (FastAPI) y la base de datos (PostgreSQL).

- **Control de Inventario y Anti Doble-gasto (RF08):** El sistema opera bajo un modelo de libro mayor (ledger). Cada entrada o salida genera un registro en la tabla INVENTORY_MOVEMENTS. Antes de autorizar una transformación, el backend verifica que la solicitud no exceda la available quantity de la tabla LOTS. Si los kilos requeridos superan las existencias, la transacción es rechazada de inmediato.
- **Idempotencia y Sincronización (Diferenciador Offline):** Para soportar la captura de datos con conectividad intermitente, el frontend genera un identificador único que viaja hasta las tablas transaccionales (RECEPTIONS, TRANSFORMATIONS) guardándose en el campo idempotency_key (marcado como Unique Key). Si la red falla y la petición se reenvía, la base de datos detecta la llave duplicada e ignora la operación, evitando registrar y descontar el inventario dos veces.
- **Gestión de Estados por Rol (RF09):** Los lotes inician su ciclo en estado "Pendiente". El sistema verifica el role del usuario en la tabla USERS. Únicamente un token autenticado con rol de "Administrador" puede transicionar un lote a "Liberado" o "Bloqueado" (guardando el motivo en status_reason). Los lotes bloqueados quedan automáticamente ocultos para las consultas de la ficha comercial pública.
- **Auditoría Inmutable (RF10):** Se implementa un registro estricto en la tabla AUDIT_EVENTS. Cada vez que un usuario altera un estado o cantidad, se guarda el actor_id, la acción realizada y, crucialmente, el estado previo (before_value) y el nuevo (after_value) en formato JSON. Esta tabla tiene bloqueadas las operaciones de eliminación (DELETE) a nivel de base de datos.
- **Cálculo Aislado de Costos (RF04, RF05, RF12):** La lógica financiera para calcular el margen y el costo unitario se centraliza en el backend, depositando los resultados en las tablas especializadas FINANCE_RECORDS y COST_ENTRIES. Esto separa la responsabilidad operativa de la financiera y permite al backend incluir controles de excepciones matemáticos (como evitar la división entre cero si una salida reporta 0 kg).

4. Flujo de Usuario y Pantallas (Prototipado)

El flujo de interacción del sistema KallpaCacao Huila está segmentado estrictamente por el rol del usuario, garantizando mediante autenticación que cada actor acceda únicamente a las funciones y datos de su competencia:

- **Panel del Operador (Enfoque de Campo):**
    - **Navegación**: El menú lateral está restringido a las operaciones de campo: Resumen, Recepción, Transformación, Trazabilidad e Inventario.

    - **Comportamiento Offline:** La interfaz principal incorpora un indicador de estado de red ("En Línea") y un gestor de sincronización que muestra los lotes pendientes de subir a FastAPI. Incluye un botón de "Sincronizar Ahora" para empujar la cola de transacciones guardadas localmente.
- **Panel del Administrador (Dashboard Gerencial):**
    - **Navegación**: Comparte la vista de estadísticas generales con el operador, pero expande el menú lateral habilitando dos apartados exclusivos y críticos: **Auditoría** y **Usuarios**.
    - **Acción Crítica**: A través de estos módulos adicionales, el administrador puede gestionar los accesos al sistema y revisar el historial inmutable de cambios (RF10).
- **Ficha Comercial (Comprador Público):**
    - **Visualización**: Interfaz pública orientada al cliente final que muestra el origen verificado del producto (ej. Finca El Porvenir, Rivera), detalles de calidad y un código QR para escaneo.
    - **Restricciones**: Cumple con la regla de negocio al indicar explícitamente que la información pública está protegida y los costos, márgenes y datos privados no son visibles.
    - **Estado del Prototipo:** En la versión actual de la interfaz, el apartado de "Recorrido certificado" (línea de tiempo) se presenta de manera estática con fines de maquetación visual.

4.1 Prototipos de interfaz

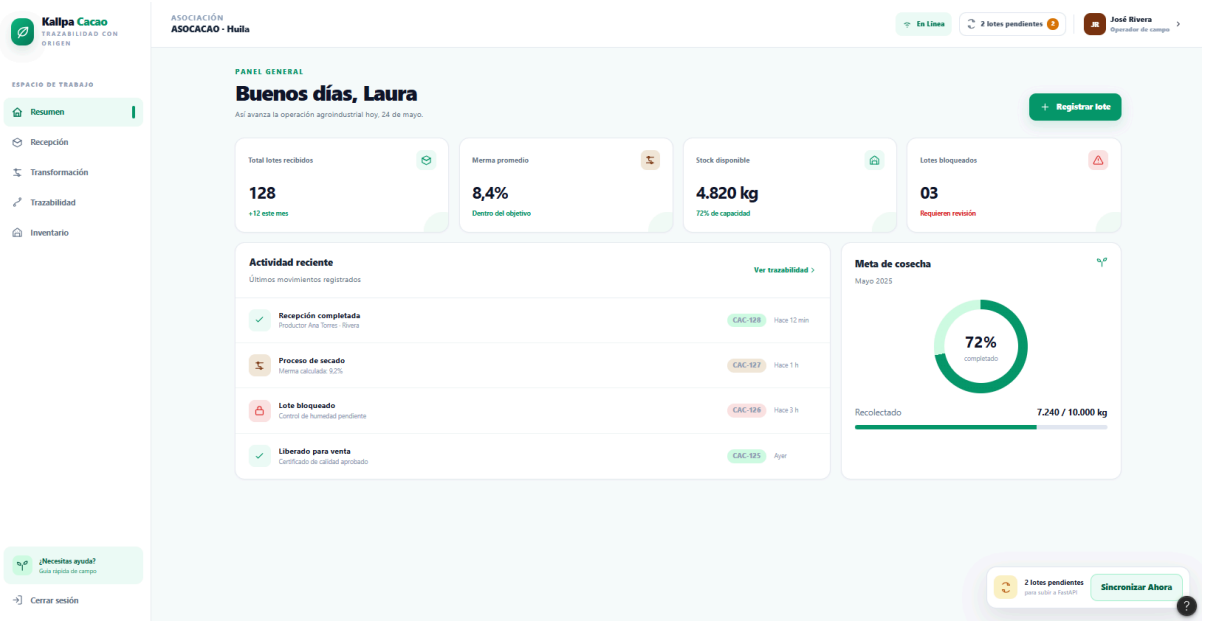


Imagen 2. (Panel Operador)

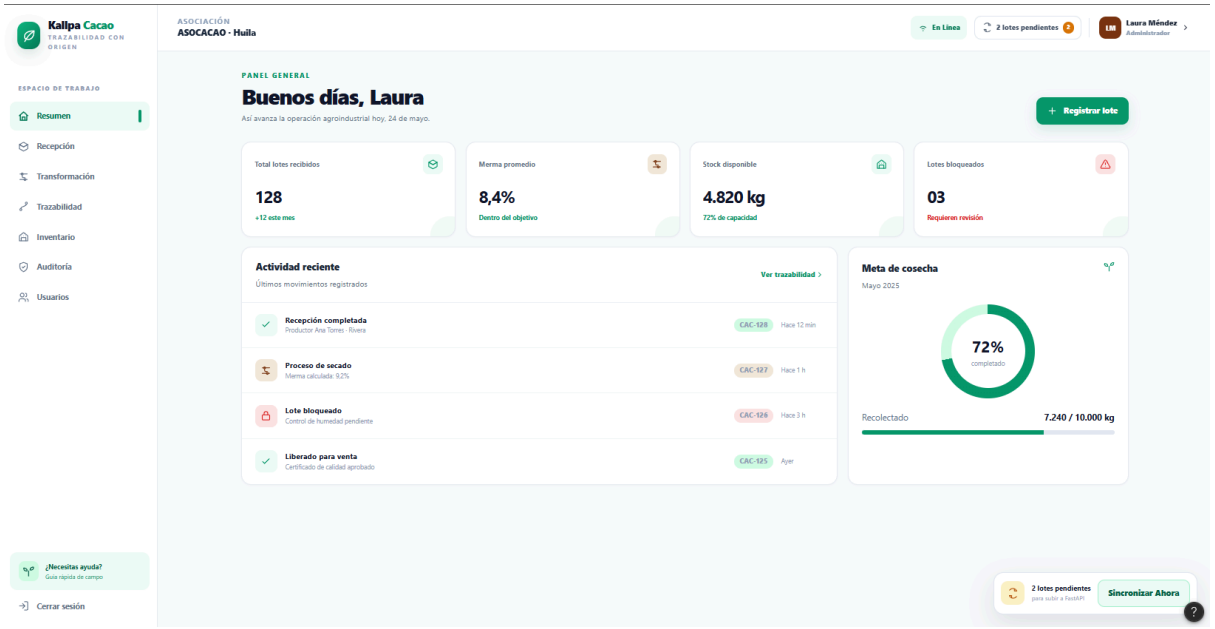


Imagen 3. (Panel Administrador)

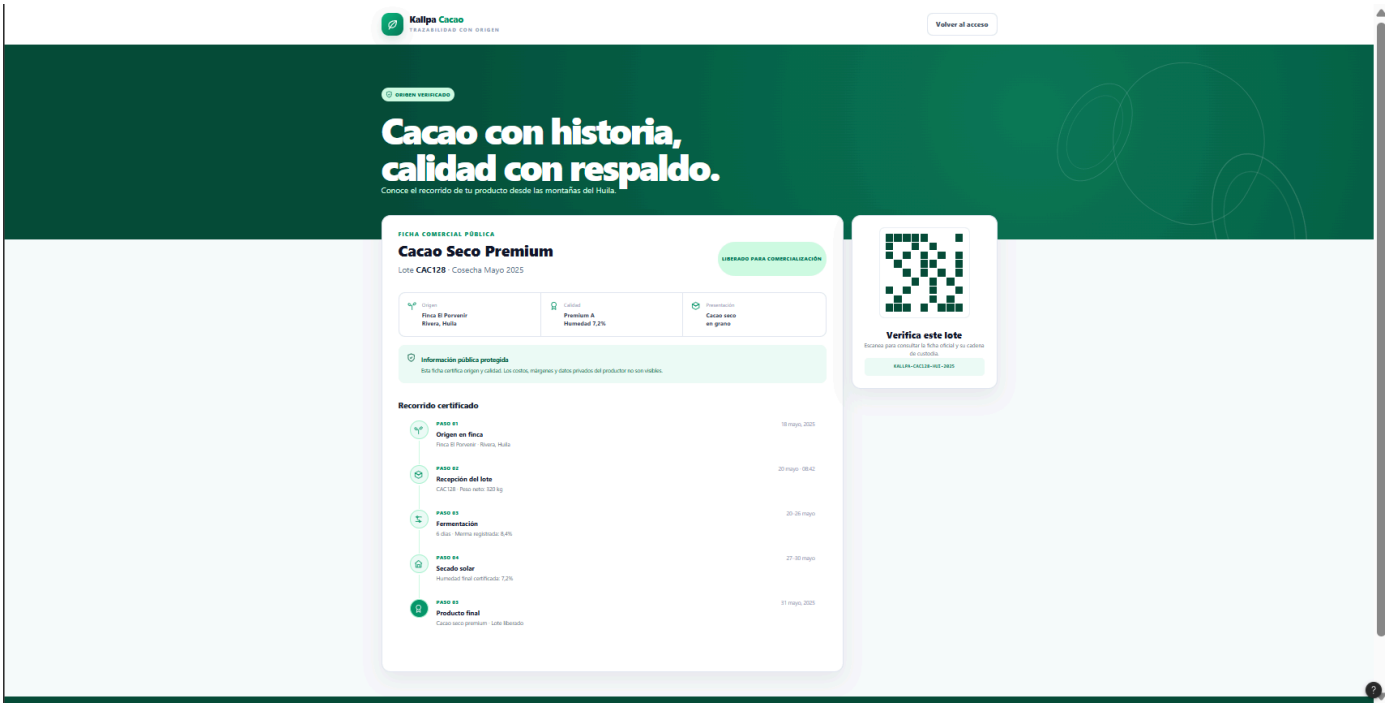


Imagen 4. (Ficha Comercial)

5. Decisiones Técnicas (Justificación y Limitaciones)

Para la construcción de este MVP, el equipo tomó las siguientes decisiones arquitectónicas documentadas para garantizar la viabilidad del proyecto en 24 horas:

- **Decisión 1: Estrategia de conectividad limitada mediante cola local (React)**
    - **Razón:** Alineado con el diferenciador elegido, las zonas rurales y plantas de beneficio en el Huila sufren de intermitencia de red. Implementar una cola local asegura que el Operador no pierda datos de recepción ni detenga la producción por fallas de internet.
    - **Alternativa Descartada:** Desarrollo de una aplicación web tradicional dependiente del servidor al 100%, la cual arrojaría errores de conexión (timeout) y pérdida de datos en formularios ante microcortes de red.

Limitación: El almacenamiento local del navegador (IndexedDB/LocalStorage) es volátil si el operador borra los datos de navegación antes de sincronizar. Exige, además, la implementación obligatoria de una llave de idempotencia (idempotency_key) en el backend para evitar registros duplicados al recuperar la señal.

- **Decisión 2: Motor Relacional (PostgreSQL) y ORM (SQLAlchemy) en el Backend**
    - **Razón:** Garantiza el cumplimiento estricto de propiedades ACID, integridad referencial y soporte nativo para restricciones únicas (idempotency_key) y tipos JSON para auditoría. SQLAlchemy facilita el control riguroso de migraciones (Alembic) y previene inyecciones SQL.
    - **Alternativa Descartada:** Bases de datos NoSQL (MongoDB/DocumentDB) o esquemas embebidos en memoria sin validación de llaves foráneas, las cuales dificultan la consistencia en transacciones financieras e inventarios.

Limitación: Requiere la ejecución de un contenedor de base de datos relacional independiente y la gestión explícita de migraciones de esquema ante cambios en los modelos de datos durante el desarrollo.